



FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

CLEYTON ENRIKE DE OLIVEIRA – RM 560485 (1TDSQ)
MATHEUS HENRIQUE NASCIMENTO DE FREITAS – RM 560442 (1TDSQ)
MATHEUS PINHEIRO ERMACORA MARTIN – RM 557720 (1TDSZ)

VIASMART - PONTO DE AJUDA NAS ESTAÇÕES

SÃO PAULO 2025

CLEYTON ENRIKE DE OLIVEIRA – RM 560485 (1TDSQ)
MATHEUS HENRIQUE NASCIMENTO DE FREITAS – RM 560442 (1TDSQ)
MATHEUS PINHEIRO ERMACORA MARTIN – RM 557720 (1TDSZ)

VIASMART - PONTO DE AJUDA NAS ESTAÇÕES

Challenge apresentado pela empresa CCR, como parte do desenvolvimento das diversas áreas de conhecimento do curso de Análise e Desenvolvimento de Sistemas.

SÃO PAULO 2025

SUMÁRIO

DESAFIO DA EQUIPE: DESENVOLVER UM TOTEM DE AJUDA PARA AUXILIAR A LOCOMOÇÃO DOS USUÁRIOS NAS ESTAÇÕES.....	4
Melhorando a experiência dos usuários.....	4
Incluir para inovar e revolucionar	5
Benefícios para a Empresa.....	5
APLICAÇÃO DE JAVA NO PROJETO	6
DESCRIÇÃO DAS FUNCIONALIDADES.....	7

DESAFIO DA EQUIPE: DESENVOLVER UM TOTEM DE AJUDA PARA AUXILIAR A LOCOMOÇÃO DOS USUÁRIOS NAS ESTAÇÕES.

Objetivo da equipe: A ideia é desenvolver um serviço de atendimento personalizado nas estações, utilizando uma assistente virtual com inteligência artificial. Estamos inovando a experiência do usuário nas estações com a implementação de um totem interativo equipado com uma assistente virtual inteligente. Essa assistente será capaz de entender as necessidades individuais de cada passageiro, oferecendo informações precisas e personalizadas sobre as melhores rotas dentro do sistema metroviário. Para garantir a inclusão, o sistema oferecerá opções de idiomas, como português e inglês, e contará com recursos de acessibilidade.

Melhorando a experiência dos usuários

Queremos poder melhorar a forma como cada usuário se locomove nas estações metroviária, sendo uma tecnologia que se adapta a cada indivíduo.

Pensando em desenvolver um produto que atenda ao máximo possível de usuários. Nosso objetivo é desenvolver uma tecnologia que auxilie de maneira prática, eficiente e inclusiva a locomoção de cada usuário até a sua estação de destino.

- **Interação intuitiva:** O passageiro se aproximaria de um ponto de atendimento equipado com uma interface fácil de usar, como um totem *touch screen*.
- **Consulta personalizada:** O totem tem um menu intuitivo que permite o usuário navegar entre as funcionalidades (Consultar rota, Informações, Ajuda), sendo a principal “Consultar rota” onde ele pode ter o auxílio da rota de estações para chegar ao seu destino.
- **Orientação detalhada:** A partir das informações fornecidas, como estação de origem e destino, a assistente geraria um mapa digital ou

um roteiro passo a passo, indicando a plataforma correta e o sentido do trem.

- **Informações adicionais:** A assistente poderia fornecer informações adicionais, como horários dos trens, tempo estimado de viagem, opções de transporte público integrado e até mesmo alertas sobre possíveis interrupções no serviço.

Incluir para inovar e revolucionar

Tendo em vista a grande diversidade usuários metroviários, com diferentes necessidades, conhecimentos e nacionalidades, estamos desenvolvendo um projeto inovador que irá revolucionar a maneira como as pessoas buscam as informações dentro das estações com o intuito de incluir de forma mais abrangente todos os tipos de usuário.

Nossa principal missão é garantir que todos os usuários tenham uma experiência única e personalizada ao interagir com o nosso totem. Para podermos alcançar esse objetivo, estamos buscando implementar uma inteligência artificial que padroniza a entrada de dados, gerada pelo usuário quando faz a interação com o totem, para um tratamento e manipulação de dados mais precisa do algoritmo responsável por fazer a busca no sistema de todas as informações e apresentar dentre elas a qual satisfaz o desejo de cada usuário.

Benefícios para a Empresa

A implementação de totens informativos nas estações proporciona uma experiência personalizada e mais satisfatória para os usuários, melhorando a percepção dos usuários pelas estações, além de criar um comprometimento com a excelência no atendimento e às necessidades individuais de cada usuário.

APLICAÇÃO DE JAVA NO PROJETO

O projeto utilizou Java para a implementação de novas classes que possibilitaram o desenvolvimento de um algoritmo de Dijkstra, responsável por encontrar a melhor rota dentro do sistema metroviário. Essa abordagem permitiu a criação de um sistema eficiente para cálculo de trajetos, garantindo que os usuários recebam informações precisas e otimizadas sobre seus deslocamentos. A estrutura orientada a objetos do Java foi essencial para modularizar o código, facilitando a manutenção e escalabilidade da aplicação.

DESCRIÇÃO DAS FUNCIONALIDADES

1. Classe Grafo()

```
1 // Classe Grafo para representar o mapa
2 public static class Grafo {
3     private final Map<String, List<String>> adjacencia;
4
5     public Grafo() {
6         adjacencia = new HashMap<>();
7     }
8
9     public void addEdge(String origem, String destino, int peso) {
10         adjacencia.putIfAbsent(origem, new ArrayList<>());
11         adjacencia.get(origem).add(destino);
12     }
13
14     public List<String> caminhoMaisCurto(String origem, String destino) {
15         // Algoritmo de busca em largura (BFS) para encontrar o caminho mais curto
16         Queue<String> fila = new LinkedList<>();
17         Map<String, String> predecessores = new HashMap<>();
18         Set<String> visitados = new HashSet<>();
19
20         fila.add(origem);
21         visitados.add(origem);
22
23         while (!fila.isEmpty()) {
24             String atual = fila.poll();
25
26             if (atual.equals(destino)) {
27                 List<String> caminho = new ArrayList<>();
28                 for (String station = destino; station != null; station = predecessores.get(station)) {
29                     caminho.add(station);
30                 }
31                 Collections.reverse(caminho);
32                 return caminho;
33             }
34
35             for (String vizinho : adjacencia.getOrDefault(atual, new ArrayList<>())) {
36                 if (!visitados.contains(vizinho)) {
37                     fila.add(vizinho);
38                     visitados.add(vizinho);
39                     predecessores.put(vizinho, atual);
40                 }
41             }
42         }
43         return Collections.emptyList(); // Se não encontrar o caminho
44     }
45 }
```

A classe Grafo é usada para representar o mapa das estações e suas conexões. Ela utiliza uma estrutura de dados do tipo Map para armazenar as adjacências entre as estações e implementa um algoritmo de busca em largura (BFS) para encontrar o caminho mais curto entre duas estações.

Funcionalidade:

1. Adição de arestas: O método `addEdge` adiciona uma conexão entre duas estações.
2. Busca do caminho mais curto: O método `caminhoMaisCurto` utiliza o algoritmo BFS para encontrar o menor caminho entre a origem e o destino.

Passos do algoritmo:

1. Inicializa uma fila para armazenar as estações a serem visitadas.
2. Usa um Map para rastrear os predecessores de cada estação.
3. Marca as estações visitadas para evitar loops.
4. Reconstrói o caminho a partir do destino até a origem, invertendo-o no final.

2. Classe consultarCaminho()

```
1 public static void consultarCaminho() {
2     String origem = JOptionPane.showInputDialog(null, "Qual estação você está?");
3     String destino = JOptionPane.showInputDialog(null, "Qual estação você quer ir?");
4
5     // Criar o grafo e adicionar as arestas
6     Grafo grafo = new Grafo();
7     grafo.addEdge("Júlio Prestes", "Palmeiras-Barra Funda", 1);
8     grafo.addEdge("Palmeiras-Barra Funda", "Júlio Prestes", 1);
9     grafo.addEdge("Palmeiras-Barra Funda", "Lapa", 1);
10    grafo.addEdge("Lapa", "Palmeiras-Barra Funda", 1);
11    grafo.addEdge("Lapa", "Domingos de Moraes", 1);
12    grafo.addEdge("Domingos de Moraes", "Lapa", 1);
13    grafo.addEdge("Domingos de Moraes", "Imperatriz Leopoldina", 1);
14    grafo.addEdge("Imperatriz Leopoldina", "Domingos de Moraes", 1);
15    grafo.addEdge("Imperatriz Leopoldina", "Presidente Altino", 1);
16    grafo.addEdge("Presidente Altino", "Imperatriz Leopoldina", 1);
17    grafo.addEdge("Presidente Altino", "Osasco", 1);
18    grafo.addEdge("Osasco", "Presidente Altino", 1);
19    grafo.addEdge("Osasco", "Comandante Sampaio", 1);
20    grafo.addEdge("Comandante Sampaio", "Osasco", 1);
21    grafo.addEdge("Comandante Sampaio", "Quitaúna", 1);
22    grafo.addEdge("Quitaúna", "Comandante Sampaio", 1);
23    grafo.addEdge("Quitaúna", "General Miguel Costa", 1);
24    grafo.addEdge("General Miguel Costa", "Quitaúna", 1);
25    grafo.addEdge("General Miguel Costa", "Carapicuíba", 1);
26    grafo.addEdge("Carapicuíba", "General Miguel Costa", 1);
27    grafo.addEdge("Carapicuíba", "Santa Terezinha", 1);
28    grafo.addEdge("Santa Terezinha", "Carapicuíba", 1);
29    grafo.addEdge("Santa Terezinha", "Antonio João", 1);
30    grafo.addEdge("Antonio João", "Santa Terezinha", 1);
31    grafo.addEdge("Antonio João", "Barueri", 1);
32    grafo.addEdge("Barueri", "Antonio João", 1);
33    grafo.addEdge("Barueri", "Jardim Belval", 1);
34    grafo.addEdge("Jardim Belval", "Barueri", 1);
35    grafo.addEdge("Jardim Belval", "Jardim Silveira", 1);
36    grafo.addEdge("Jardim Silveira", "Jardim Belval", 1);
37    grafo.addEdge("Jardim Silveira", "Jandira", 1);
38    grafo.addEdge("Jandira", "Jardim Silveira", 1);
39    grafo.addEdge("Jandira", "Sagrado Coração", 1);
40    grafo.addEdge("Sagrado Coração", "Jandira", 1);
41    grafo.addEdge("Sagrado Coração", "Engenheiro Cardoso", 1);
42    grafo.addEdge("Engenheiro Cardoso", "Sagrado Coração", 1);
43    grafo.addEdge("Engenheiro Cardoso", "Itapevi", 1);
44    grafo.addEdge("Itapevi", "Engenheiro Cardoso", 1);
45    grafo.addEdge("Itapevi", "Amador Bueno", 1);
46    grafo.addEdge("Amador Bueno", "Itapevi", 1);
47
48    // Encontrar o caminho mais curto
49    List<String> caminhoMaisCurto = grafo.caminhoMaisCurto(origem, destino);
50
51    // Exibir o caminho
52    if (caminhoMaisCurto.isEmpty()) {
53        JOptionPane.showMessageDialog(null, "Não foi possível encontrar o caminho entre as estações.");
54    } else {
55        JOptionPane.showMessageDialog(null, "Caminho mais curto: " + caminhoMaisCurto);
56    }
57 }
```

Este método é responsável por interagir com o usuário, coletar as estações de origem e destino, e exibir o caminho mais curto entre elas.

Funcionalidade:

- Entrada do usuário: Usa JOptionPane para solicitar ao usuário a estação de origem e destino.
- Construção do grafo: Adiciona manualmente as conexões entre as estações.
- Busca e exibição do caminho: Chama o método caminhoMaisCurto da classe Grafo e exibe o resultado em uma caixa de diálogo.

3. Classe caminhoMaisCurto()

```
1 public List<String> caminhoMaisCurto(String origem, String destino) {
2     // Mapa para armazenar a distância mínima de cada estação até a origem
3     Map<String, Integer> distancias = new HashMap<>();
4     // Mapa para armazenar o caminho (anterior) de cada estação
5     Map<String, String> predecessores = new HashMap<>();
6     // Mapa para armazenar as estações não visitadas
7     Set<String> naoVisitadas = new HashSet<>();
8
9     // Inicializa as distâncias com infinito e a origem com distância 0
10    for (String estacao : grafo.keySet()) {
11        distancias.put(estacao, Integer.MAX_VALUE);
12        naoVisitadas.add(estacao);
13    }
14    distancias.put(origem, 0);
15
16    // Enquanto houver estações não visitadas
17    while (!naoVisitadas.isEmpty()) {
18        // Estação com a menor distância (a mais próxima)
19        String atual = null;
20        int menorDistancia = Integer.MAX_VALUE;
21        for (String estacao : naoVisitadas) {
22            int distancia = distancias.get(estacao);
23            if (distancia < menorDistancia) {
24                menorDistancia = distancia;
25                atual = estacao;
26            }
27        }
28
29        // Se a estação atual for a estação de destino, terminou o algoritmo
30        if (atual.equals(destino)) {
31            break;
32        }
33
34        // Remover a estação atual da lista de não visitadas
35        naoVisitadas.remove(atual);
36
37        // Para cada vizinho da estação atual
38        for (Map.Entry<String, Integer> vizinho : grafo.get(atual).entrySet()) {
39            String vizinhoEstacao = vizinho.getKey();
40            int peso = vizinho.getValue();
41
42            // Se a distância através da estação atual for menor que a distância já registrada
43            int novaDistancia = distancias.get(atual) + peso;
44            if (novaDistancia < distancias.get(vizinhoEstacao)) {
45                distancias.put(vizinhoEstacao, novaDistancia);
46                predecessores.put(vizinhoEstacao, atual);
47            }
48        }
49    }
50
51    // Reconstruir o caminho a partir do destino até a origem
52    List<String> caminho = new ArrayList<>();
53    String atual = destino;
54    while (atual != null) {
55        caminho.add(atual);
56        atual = predecessores.get(atual);
57    }
58
59    // Se o caminho for encontrado, inverte o caminho para origem -> destino
60    Collections.reverse(caminho);
61    return caminho.size() == 1 && !caminho.get(0).equals(origem) ? new ArrayList<>() : caminho; // Retorna lista vazia se não houver caminho
62 }
```

O método `caminhoMaisCurto` é o coração do código. Ele calcula o caminho mais curto entre duas estações usando o algoritmo de Dijkstra, que é amplamente utilizado para encontrar rotas com o menor custo (neste caso, a menor distância) em grafos.

Passos do Algoritmo:

1. Inicialização:

- Um mapa (distancias) armazena a distância mínima de cada estação até a origem.
- Um mapa (predecessores) armazena o caminho percorrido (a estação anterior no caminho mais curto).
- Um conjunto (naoVisitadas) armazena todas as estações ainda não processadas.

2. Definição das Distâncias:

- Todas as estações começam com distância infinita (Integer.MAX_VALUE), exceto a estação de origem, que tem distância 0.

3. Processamento das Estações:

- Enquanto houver estações não visitadas, o algoritmo seleciona a estação com a menor distância atual (atual).
- Para cada vizinho da estação atual, o algoritmo calcula uma nova distância (novaDistancia) somando a distância da estação atual com o peso da conexão até o vizinho.
- Se a novaDistancia for menor que a distância registrada para o vizinho, a distância e o predecessor são atualizados.

4. Reconstrução do Caminho:

- Após encontrar o destino, o caminho é reconstruído a partir do destino até a origem, utilizando o mapa de predecessores.
- O caminho é invertido para exibir a rota na ordem correta (origem → destino).

5. Resultado:

- Se o caminho for encontrado, ele é retornado como uma lista de estações.
- Caso contrário, uma lista vazia é retornada.

3. Classe Util()

```
1 // Método para calcular o caminho mais curto usando o algoritmo de Dijkstra
2 public static List<String> shortestPath(Grafo grafo, String origem, String destino) {
3     // Dijkstra - Inicializa a tabela de distâncias
4     Map<String, Integer> distancias = new HashMap<>();
5     Map<String, String> antecessores = new HashMap<>();
6     Set<String> visitados = new HashSet<>();
7     PriorityQueue<String> filaPrioridade = new PriorityQueue<>(Comparator.comparingInt(distancias::get));
8
9     // Inicializa todas as distâncias como infinito, exceto a origem
10    for (String estacao : grafo.getConexoes(origem).keySet()) {
11        distancias.put(estacao, Integer.MAX_VALUE);
12    }
13    distancias.put(origem, 0);
14    filaPrioridade.add(origem);
15
16    // Algoritmo de Dijkstra
17    while (!filaPrioridade.isEmpty()) {
18        String estacaoAtual = filaPrioridade.poll();
19
20        if (estacaoAtual.equals(destino)) {
21            break; // Encontrou o destino
22        }
23
24        if (visitados.contains(estacaoAtual)) {
25            continue;
26        }
27
28        visitados.add(estacaoAtual);
29
30        for (Map.Entry<String, Integer> vizinho : grafo.getConexoes(estacaoAtual).entrySet()) {
31            String proximaEstacao = vizinho.getKey();
32            int distancia = vizinho.getValue();
33            int novaDistancia = distancias.get(estacaoAtual) + distancia;
34
35            if (novaDistancia < distancias.getOrDefault(proximaEstacao, Integer.MAX_VALUE)) {
36                distancias.put(proximaEstacao, novaDistancia);
37                antecessores.put(proximaEstacao, estacaoAtual);
38                filaPrioridade.add(proximaEstacao);
39            }
40        }
41    }
42
43    // Reconstruir o caminho mais curto
44    List<String> caminho = new ArrayList<>();
45    String atual = destino;
46    while (atual != null) {
47        caminho.add(atual);
48        atual = antecessores.get(atual);
49    }
50    Collections.reverse(caminho); // Reverte o caminho para ficar do começo ao fim
51
52    return caminho.size() == 1 ? Collections.emptyList() : caminho;
53 }
54
```

O método `shortestPath` é o núcleo do código. Ele calcula o caminho mais curto entre duas estações usando o algoritmo de Dijkstra, que é amplamente utilizado para encontrar rotas com o menor custo (neste caso, a menor distância) em grafos.

Passos do Algoritmo:

1.Inicialização:

- Um mapa (`distancias`) armazena a distância mínima de cada estação até a origem.
- Um mapa (`antecessores`) armazena o caminho percorrido (a estação anterior no caminho mais curto).
- Um conjunto (`visitados`) armazena as estações já processadas.
- Uma fila de prioridade (`filaPrioridade`) ordena as estações pela distância.

2.Definição das Distâncias:

- Todas as estações começam com distância infinita (`Integer.MAX_VALUE`), exceto a estação de origem, que tem distância 0.

3.Processamento das Estações:

- Enquanto houver estações na fila de prioridade, o algoritmo seleciona a estação com a menor distância atual (`estacaoAtual`).
- Para cada vizinho da estação `estacaoAtual`, o algoritmo calcula uma nova distância (`novaDistancia`) somando a distância da estação `estacaoAtual` com o peso da conexão até o vizinho.
- Se a `novaDistancia` for menor que a distância registrada para o vizinho, a distância e o predecessor são atualizados, e o vizinho é adicionado à fila de prioridade.

4.Reconstrução do Caminho:

- Após encontrar o destino, o caminho é reconstruído a partir do destino até a origem, utilizando o mapa de predecessores.
- O caminho é invertido para exibir a rota na ordem correta (origem → destino).

5.Resultado:

- Se o caminho for encontrado, ele é retornado como uma lista de estações.
- Caso contrário, uma lista vazia é retornada.

TELAS DO PROTÓTIPO

1.Tela Menu



```
1 public static void menuPrincipal() {
2     String opcao = JOptionPane.showInputDialog("Menu Principal: 1 - Consultar caminho mais curto/2 - Informações sobre lotação de trem/3 - Dúvidas frequentes/4 - Cadastrar linha/5 - Encerrar atendimento", "Menu", JOptionPane.QUESTION_MESSAGE);
3
4     while (opcao.charAt(0) != '\0') {
5         opcao = JOptionPane.showInputDialog("Opção inválida. Escolha uma opção entre 1 e 5.", "Erro", JOptionPane.ERROR_MESSAGE);
6     }
7
8     switch (opcao) {
9         case "1":
10             Caminho.consultarCaminho(); // Chama o método na classe Caminho
11             break;
12         case "2":
13             Lotacao.mostrarLotacao(); // Chama o método na classe Lotacao
14             break;
15         case "3":
16             DúvidasFrequentes.mostrarDúvidasFrequentes(); // Chama o método na classe DúvidasFrequentes
17             break;
18         case "4":
19             try {
20                 Linha.linha(); // Chama o método para inserir linha
21             } catch (SQLException | ClassNotFoundException e) {
22                 e.printStackTrace();
23             }
24             break;
25         case "5":
26             Encerrar(); // Chama o método para encerrar o programa
27             break;
28     }
29 }
```

A tela menu permite que o usuário navegue pelo sistema, podendo escolher qual funcionalidade ele deseja consultar. No caso do projeto, a principal funcionalidade é a “Consultar Rota”.

2.Tela Consultar Rota

Voltar

Consultar **rota**

Sua estação atual

Vila Mariana

Sua estação final

Bresser - Mooca

Mostrar Rota

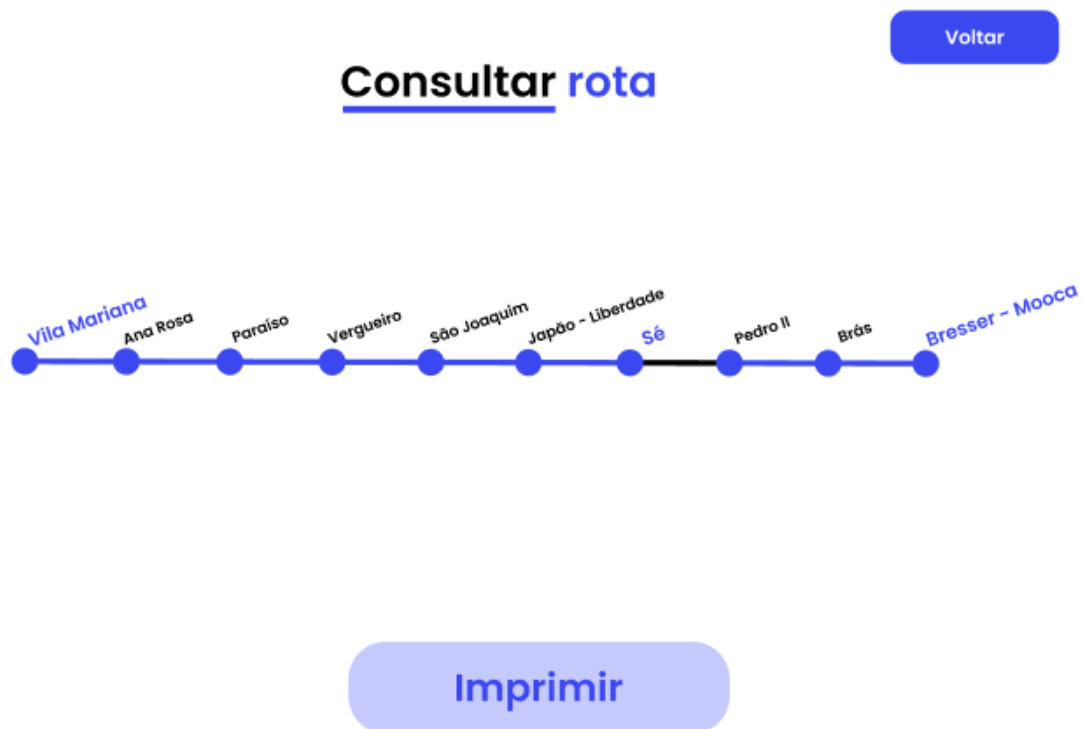
```

1  public static void consultarCaminho() {
2      String origem = JOptionPane.showInputDialog(null, "Qual estação você está?");
3      String destino = JOptionPane.showInputDialog(null, "Qual estação você quer ir?");
4
5      // Criar o grafo e adicionar as arestas
6      Grafo grafo = new Grafo();
7      grafo.addEdge("Júlio Prestes", "Palmeiras-Barra Funda", 1);
8      grafo.addEdge("Palmeiras-Barra Funda", "Júlio Prestes", 1);
9      grafo.addEdge("Palmeiras-Barra Funda", "Lapa", 1);
10     grafo.addEdge("Lapa", "Palmeiras-Barra Funda", 1);
11     grafo.addEdge("Lapa", "Domingos de Moraes", 1);
12     grafo.addEdge("Domingos de Moraes", "Lapa", 1);
13     grafo.addEdge("Domingos de Moraes", "Imperatriz Leopoldina", 1);
14     grafo.addEdge("Imperatriz Leopoldina", "Domingos de Moraes", 1);
15     grafo.addEdge("Imperatriz Leopoldina", "Presidente Altino", 1);
16     grafo.addEdge("Presidente Altino", "Imperatriz Leopoldina", 1);
17     grafo.addEdge("Presidente Altino", "Osasco", 1);
18     grafo.addEdge("Osasco", "Presidente Altino", 1);
19     grafo.addEdge("Osasco", "Comandante Sampaio", 1);
20     grafo.addEdge("Comandante Sampaio", "Osasco", 1);
21     grafo.addEdge("Comandante Sampaio", "Quitaúna", 1);
22     grafo.addEdge("Quitaúna", "Comandante Sampaio", 1);
23     grafo.addEdge("Quitaúna", "General Miguel Costa", 1);
24     grafo.addEdge("General Miguel Costa", "Quitaúna", 1);
25     grafo.addEdge("General Miguel Costa", "Carapicuíba", 1);
26     grafo.addEdge("Carapicuíba", "General Miguel Costa", 1);
27     grafo.addEdge("Carapicuíba", "Santa Terezinha", 1);
28     grafo.addEdge("Santa Terezinha", "Carapicuíba", 1);
29     grafo.addEdge("Santa Terezinha", "Antonio João", 1);
30     grafo.addEdge("Antonio João", "Santa Terezinha", 1);
31     grafo.addEdge("Antonio João", "Barueri", 1);
32     grafo.addEdge("Barueri", "Antonio João", 1);
33     grafo.addEdge("Barueri", "Jardim Belval", 1);
34     grafo.addEdge("Jardim Belval", "Barueri", 1);
35     grafo.addEdge("Jardim Belval", "Jardim Silveira", 1);
36     grafo.addEdge("Jardim Silveira", "Jardim Belval", 1);
37     grafo.addEdge("Jardim Silveira", "Jandira", 1);
38     grafo.addEdge("Jandira", "Jardim Silveira", 1);
39     grafo.addEdge("Jandira", "Sagrado Coração", 1);
40     grafo.addEdge("Sagrado Coração", "Jandira", 1);
41     grafo.addEdge("Sagrado Coração", "Engenheiro Cardoso", 1);
42     grafo.addEdge("Engenheiro Cardoso", "Sagrado Coração", 1);
43     grafo.addEdge("Engenheiro Cardoso", "Itapevi", 1);
44     grafo.addEdge("Itapevi", "Engenheiro Cardoso", 1);
45     grafo.addEdge("Itapevi", "Amador Bueno", 1);
46     grafo.addEdge("Amador Bueno", "Itapevi", 1);
47
48     // Encontrar o caminho mais curto
49     List<String> caminhoMaisCurto = grafo.caminhoMaisCurto(origem, destino);
50
51     // Exibir o caminho
52     if (caminhoMaisCurto.isEmpty()) {
53         JOptionPane.showMessageDialog(null, "Não foi possível encontrar o caminho entre as estações.");
54     } else {
55         JOptionPane.showMessageDialog(null, "Caminho mais curto: " + caminhoMaisCurto);
56     }
57 }

```

O usuário irá digitar a estação de origem e destino na tela do totem, o algoritmo irá receber a informação e assim executará o algoritmo de Dijkstra para encontrar o melhor caminho que usuário deverá percorrer para chegar ao seu destino.

3.Tela Rota



Na tela de rota, será retornado a busca realizada pelo algoritmo de trajeto que o usuário deverá percorrer para chegar em sua estação de destino.